Advanced Computer Networks - CS6L048

Split-Execution of Heavy Tasks in a Decentralized System

Ravilisetty Makarandh	22CS01002
Soham Chakraborty	22CS02002
Gontla Sai Venkata Pramod	22CS01033
Bandaru Lavadeep	22CS01049
Jagana Deepika	22CS01036

Term Project Report

April 23, 2026

Contents

1 Problem Statement **2**

2 System Overview **3**
2.1 Specification 3
2.2 Processing of Tasks 4

3 Code Execution **5**
3.1 Commands 5
3.2 Input Format 5
3.3 Output 6

4 Results **8**

5 Conclusions **10**

1 Problem Statement

Consider a set of N devices $D_1, D_2, \ldots, D_N$ forming a mesh network, randomly deployed over an area of size $A \times A$. Each device has a fixed communication radius R; two devices are neighbors if and only if their Manhattan distance is at most R.

A heavy computation task is divided into K components $C_1, C_2, \ldots, C_K$. Each component is installed on exactly one device (the assignment is initially uniform, i.e., each component is assigned to exactly N/K devices in a round-robin fashion, then the mapping is randomly shuffled). To complete execution, a task must be processed by all K components *in sequence*:

$$C_1 \to C_2 \to \cdots \to C_K \to \text{return to origin}$$

After being processed by component C_i, the task is forwarded through the mesh network (via flooding) to the device hosting component C_{i+1}.

Tasks are generated non-uniformly across the network area. Different cells of the $A \times A$ grid generate tasks at different rates, with arrival counts at each cell (i, j) drawn from a Poisson distribution with spatially varying intensity λ_{ij}.

2 System Overview

2.1 Specification

- The deployment area is of size A, divided into an $N \times N$ grid of unit cells.
- Events are generated at these grid cells following a Poisson arrival process.
- Since event arrival is **non-uniform**, each cell (i, j) has its own Poisson parameter λ_{ij}. The number of events at cell (i, j) in a given time slot is drawn from Poisson(λ_{ij}), allowing arrival intensity to vary across the area.
- Some of the grid cells host *devices*. There are M *devices* $D_1, D_2, \ldots, D_M$.
- Two devices D_i and D_j can communicate directly if their Manhattan distance satisfies:
$$|x_i - x_j| + |y_i - y_j| \leq R$$
where R is the communication radius shared by all devices.
- Each arriving event generates a *task*.
- Each task is divided into K components $C_1, C_2, \ldots, C_K$, which must be processed in sequence.
- Each *device* can process only one of the K components.
- Component distribution is **uniform**: each component C_k is assigned to exactly $\lfloor M/K \rfloor$ of the M devices.
- Consider the below example a with 4×4 grid, 6 *devices* and each task is divided into 3 components.

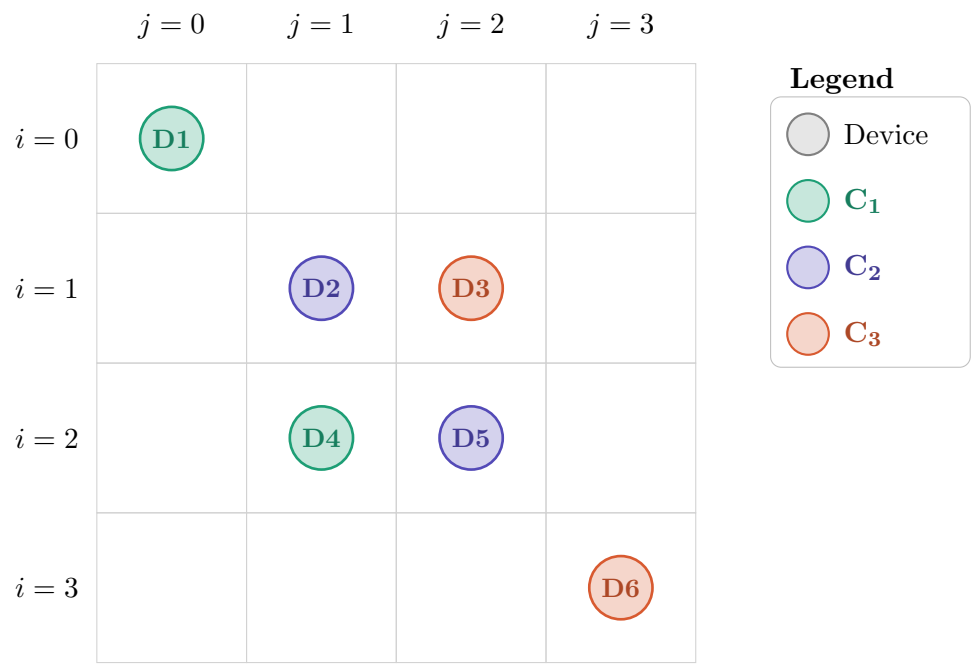

Figure 1: 4×4 deployment grid. Circles represent devices; fill color encodes the hosted component (C_1, C_2, C_3).

- Below is an image depicting the practical scenario of a non-uniform event arrival density, and how it can be converted into grid cells with different parameters for Poisson distribution.

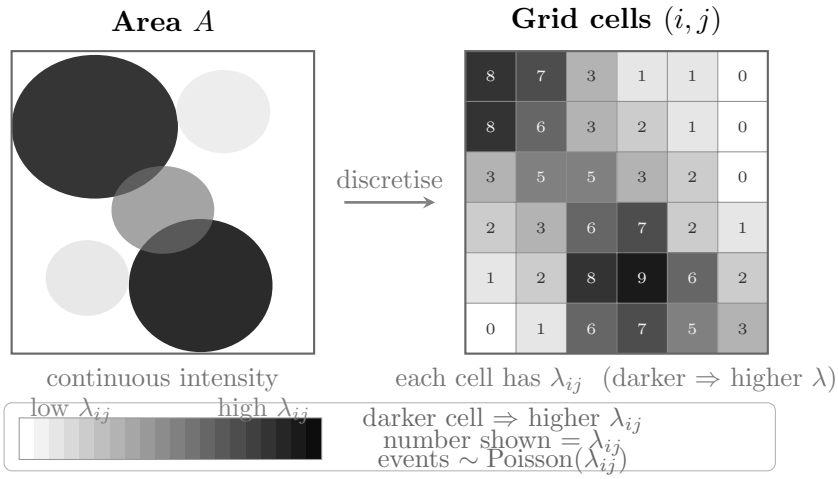

Figure 2: Discretisation of the deployment area into an $N \times N$ grid. The continuous spatial intensity (left) can be approximated by assigning a Poisson parameter λ_{ij} to each cell (i, j) (right), where darker shading indicates a higher arrival rate.

2.2 Processing of Tasks

- When an event is generated at cell (i, j), it is picked up by all devices within communication radius R of that cell.
- Simulation time is divided into fixed intervals. Each interval consists of two phases:
  - **Computation phase:** If a device hosts the component required by the task, it processes the task. The speed of processing depends on the service rate μ_k of that component.
  - **Signal phase:** All inter-device communication and message forwarding takes place.
- Once all K components of a task have been processed in sequence $C_1 \to C_2 \to \cdots \to C_K$, the task returns to its origin cell.

3 Code Execution

The code can be found at this GitHub repository:
https://github.com/rmak05/Decentralized_Split-Execution_of_Heavy_Tasks

3.1 Commands

- Recommended platform: **Windows**.
- Compile and run:

Step	Command
Compile	`g++ main.cpp -o program.exe`
Run	`./program.exe`
Run (live mode)	`./program.exe --live`

3.2 Input Format

- T — number of test cases, components are randomly and uniformly distributed over the given devices, and test cases are generated automatically.
- N — side length of the deployment area (grid is $N \times N$).
- R — communication radius of each device; two devices are neighbors if their Manhattan distance is $\leq R$.
- M — total number of devices. Must be a multiple of K.
- K — number of components per task.
- `computation_period` (milliseconds) — duration of each computation phase per simulation cycle.
- `signal_period` (milliseconds) — duration of each message-forwarding (signal) phase per simulation cycle.
- `simulation_time` (seconds) — total time for which new tasks are generated.
- $D[1 \ldots M]$ — locations of the M devices as (x, y) coordinates on the $N \times N$ grid (0-based coordinates).
- $\mu[1 \ldots K]$ — service rate of each component C_i in task components per second.
- $\lambda[i][j]$ — task arrival rate at grid cell (i, j) for $0 \leq i, j < N$.

The *input file* should look like this:

T
N
R
M
K
`computation_period`
`signal_period`
`simulation_time`
$D_1.x$ $D_1.y$
$D_2.x$ $D_2.y$
$\ldots$
$D_M.x$ $D_M.y$
μ_1
μ_2
$\ldots$
μ_K
$\lambda_{0,0}$ $\lambda_{0,1}$ $\ldots \lambda_{0,N-1}$
$\lambda_{1,0}$ $\lambda_{1,1}$ $\ldots \lambda_{1,N-1}$
$\ldots$
$\lambda_{N-1,0}$ $\lambda_{N-1,1}$ $\ldots \lambda_{N-1,N-1}$

3.3 Output

- Components are randomly and uniformly distributed over the given devices, and test cases are generated automatically.
- For each test case run:
  - Queue lengths at each device are displayed live during the simulation (when run with `--live`).
  - At the end of the run, the following summary metrics are reported:
    - Average queue length L
    - Average time per task T (in ms)
- Below are some snapshots:

```
-------------------------------------------------------------
Sample : 1

Current State - COMPUTATION
   Device 1:  (0,0)  Component 3    -   Queue length = 8
   Device 2:  (1,1)  Component 3    -   Queue length = 14
   Device 3:  (2,2)  Component 1    -   Queue length = 11
   Device 4:  (3,3)  Component 2    -   Queue length = 5
   Device 5:  (1,2)  Component 1    -   Queue length = 17
   Device 6:  (2,1)  Component 2    -   Queue length = 12
```

Figure 3: Live Simulation while running

```
-------------------------------------------------------------
Sample : 1

New Event Generation stopped.

Current State - SIGNAL
   Device 1:  (0,0)  Component 2    -   Queue length = 0
   Device 2:  (1,1)  Component 2    -   Queue length = 0
   Device 3:  (2,2)  Component 3    -   Queue length = 0
   Device 4:  (3,3)  Component 1    -   Queue length = 0
   Device 5:  (1,2)  Component 3    -   Queue length = 0
   Device 6:  (2,1)  Component 1    -   Queue length = 0

Average Time per task T  = 141.791 ms
Average Queue Length  L  = 11.964
=============================================================
```

Figure 4: Live Simulation after termination

```
-------------------------------------------------------------
Sample : 1

Average Time per task T  = 159.42 ms
Average Queue Length  L  = 10.5682
=============================================================
-------------------------------------------------------------
Sample : 2

Average Time per task T  = 146.667 ms
Average Queue Length  L  = 12.9583
=============================================================
-------------------------------------------------------------
Sample : 3

Average Time per task T  = 195.312 ms
Average Queue Length  L  = 10.1187
=============================================================
```

Figure 5: Default Simulation

4 Results

The below results have been obtained by running the testcase in Figure 1 against various parameters.

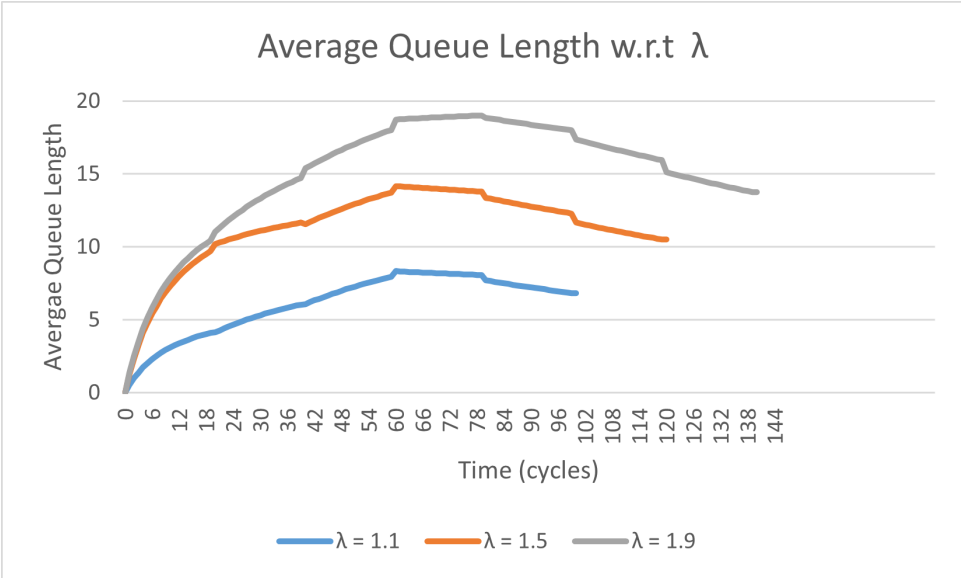

Figure 6: Queue length is plotted for different average values of Poisson parameter λ.

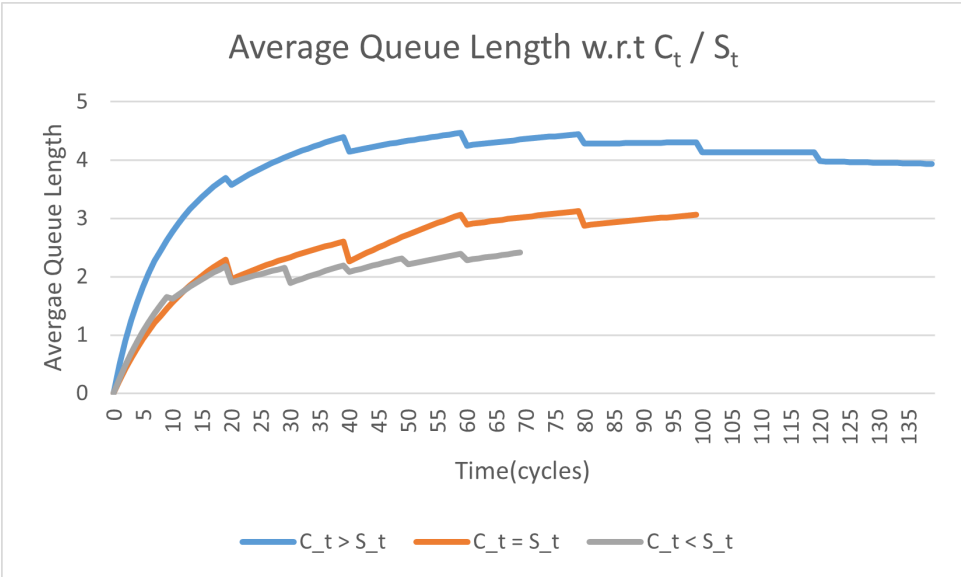

Figure 7: Queue length is plotted for different inequalities of Computation Time(C_t) and Signal Time(S_t).

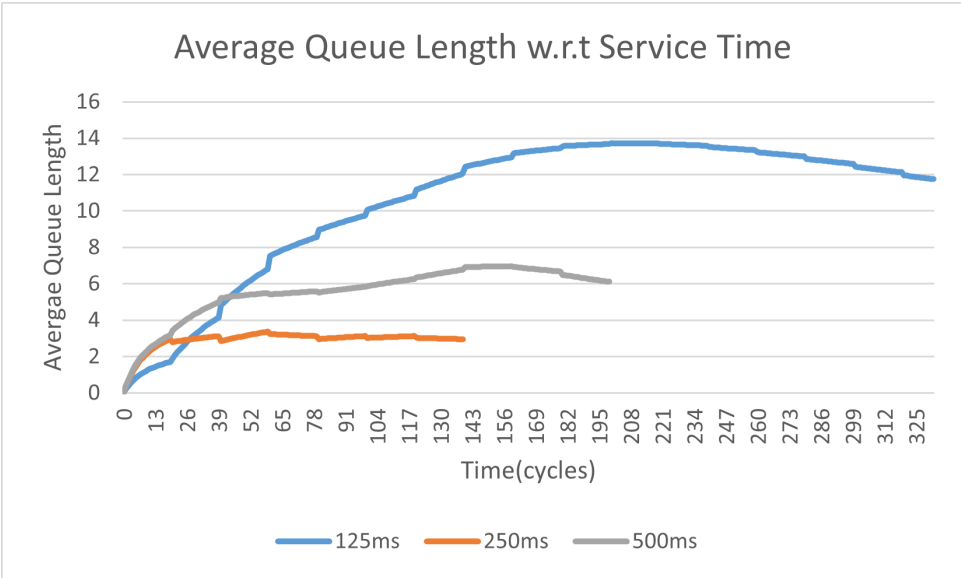

Figure 8: Queue length is plotted for different service times.

In Figure 9, events are being generated at only of the grid cells and the corresponding total execution times are displayed.

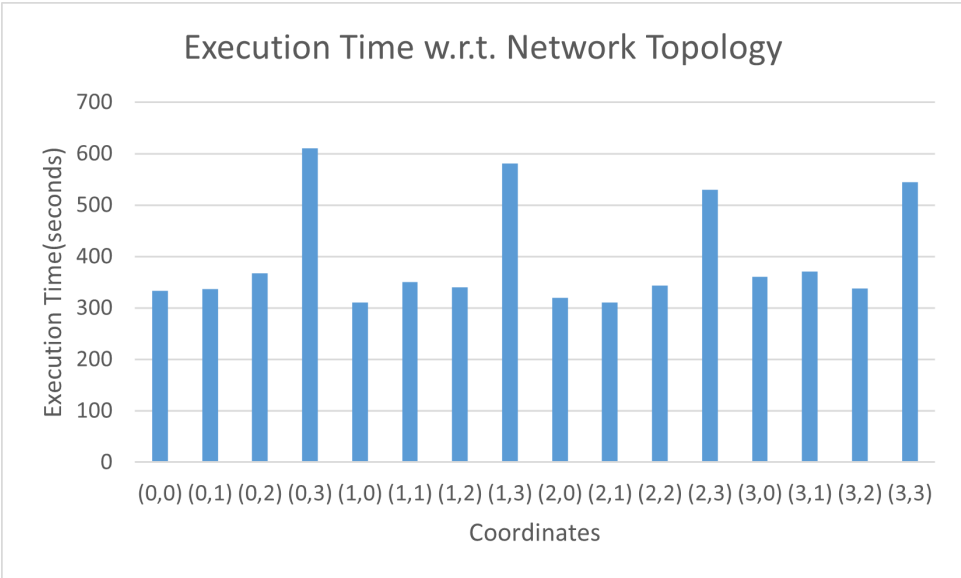

Figure 9: Total execution time is plotted for different locations in the grid.

5 Conclusions

- **Effect of Arrival Rate (λ):** As the task arrival rate λ increases, both the average queue length L and the average task completion time T increase monotonically. This is consistent with queueing theory — higher arrival rates reduce the gap between arrival and service rates, causing tasks to accumulate in queues and wait longer before being processed.
- **Computation Period vs. Signal Period:** Configurations where the signal period exceeds the computation period outperform the reverse. When computation time dominates signaling, nodes process tasks using stale or incomplete network information, since updates do not propagate sufficiently in the flooding phase. This leads to poor load balancing, causing some nodes to become congested while others remain underutilized. As a result, queue lengths increase and task completion times degrade. Hence, insufficient signaling reduces overall system efficiency despite higher local computation.
- **Non-Monotonic Effect of Service Rate:** A service rate of 250 ms yielded lower task completion times than both 125 ms and 500 ms. A very slow service rate (500 ms) causes tasks to accumulate faster than they are processed, resulting in large queues and high latency. A very fast rate (125 ms) introduces overhead from excessively frequent computation ticks relative to the simulation frames, causing idle cycles. The intermediate rate of 250 ms balances throughput and scheduling overhead, achieving optimal performance.
- **Effect of Spatial Distance from Signal Origin:** Tasks generated farther from the devices hosting the required components incur higher end-to-end latency. Since the network uses flood-based forwarding over a multi-hop mesh, greater Manhattan distance between the task origin and the target component implies more hops, longer propagation delays during the signal phase, and increased probability of queue contention at intermediate nodes — all of which collectively raise execution time.